

Practical 1

Practical Aim: To implement fundamental array operations.

Problem 1

A bakery prepares n items every morning and places them in a display row. At the end of each hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the front. Given the initial row and the number of hours h , print the final display order. Describe the approach you used. How does it behave when h is very large, say 10 million? Is there a way to get the correct result without performing the operation h times?

Code:

```
import java.util.Scanner;

public class Problem1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

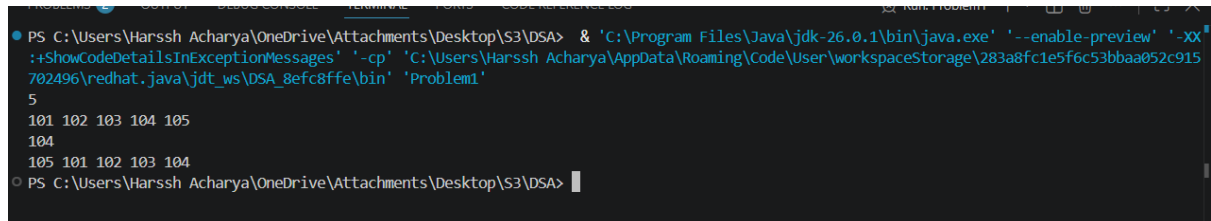
        int n = sc.nextInt();
        String[] items = new String[n];

        for (int i = 0; i < n; i++) {
            items[i] = sc.next();
        }

        int h = sc.nextInt();
        int k = h % n;

        for (int i = 0; i < n; i++) {
            System.out.print(items[(i + k) % n] + " ");
        }
    }
}
```

OUTPUT:



```
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\DSA> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Harssh Acharya\AppData\Roaming\Code\User\workspaceStorage\283a8fc1e5f6c53bbaa052c915702496\redhat.java\jdt_ws\DSA_8efc8ffe\bin' 'Problem1'
5
101 102 103 104 105
104
105 101 102 103 104
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\DSA>
```

Conclusion:

The array is rotated by moving the first element to the end. For very large h , we can use $h \% n$ to reduce the number of rotations, making the solution faster.

Problem 2

A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs. Describe the approach you used. How many times does your solution scan the data? If the library had 100,000 borrow records, would your approach still be practical? Can you think of a way that requires fewer passes?

Code:

```
import java.util.*;

public class Problem2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of borrow records: ");
        int n = sc.nextInt();

        int[] books = new int[n];

        System.out.println("Enter Book IDs:");
        for (int i = 0; i < n; i++) {
            books[i] = sc.nextInt();
        }

        System.out.println("Books borrowed more than once:");

        for (int i = 0; i < n; i++) {
            boolean printed = false;

            for (int k = 0; k < i; k++) {
                if (books[i] == books[k]) {
                    printed = true;
                    break;
                }
            }

            if (printed)
                continue;

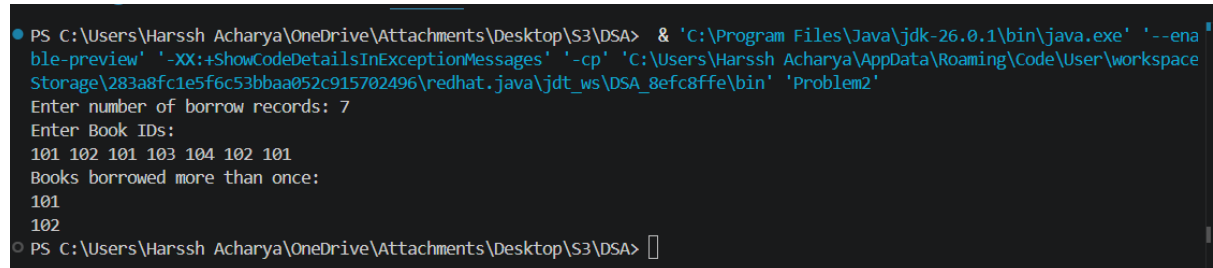
            int count = 0;
```

```
        for (int j = 0; j < n; j++) {
            if (books[i] == books[j]) {
                count++;
            }
        }

        if (count > 1) {
            System.out.println(books[i]);
        }
    }

    sc.close();
}
}
```

OUTPUT:



Conclusion:

This program finds the books that were borrowed more than once. It counts each book ID and prints it only once if its count is greater than 1.

Problem 3

A school newspaper editor wants to highlight the most impressive word from a submitted article on the front page. The word is chosen simply by length — the longest one wins. Given a sentence, print the winning word and how many letters it has. Describe the approach you used. What does your solution do when two words have the same length? Is that behavior intentional, and can you think of a way to handle it explicitly?

Code:

```
import java.util.*;

public class Problem3 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter a sentence:");
```

```
String sentence = sc.nextLine();

String[] words = sentence.split(" ");

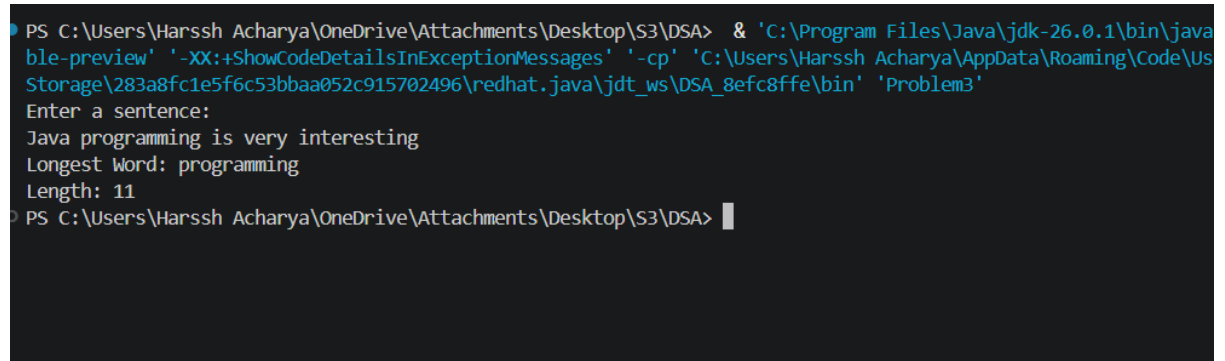
String longest = words[0];

for (int i = 1; i < words.length; i++) {
    if (words[i].length() > longest.length()) {
        longest = words[i];
    }
}

System.out.println("Longest Word: " + longest);
System.out.println("Length: " + longest.length());

sc.close();
}
```

OUTPUT:



```
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\DSA> & 'C:\Program Files\Java\jdk-26.0.1\bin\java
ble-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Harssh Acharya\AppData\Roaming\Code\Us
Storage\283a8fc1e5f6c53bbaa052c915702496\redhat.java\jdt_ws\DSA_8efc8ffe\bin' 'Problem3'
Enter a sentence:
Java programming is very interesting
Longest Word: programming
Length: 11
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\DSA> |
```

Conclusion :

This program finds the longest word in a sentence by comparing the length of each word. It then prints the longest word and its length.

Github link: <https://github.com/haarshaacharya/FDSA/tree/main/Prac1>